MidEng GK8.2 Document Oriented Middleware using MongoDB

Verfasser: Ramis Ekici

Datum: 01.04.2026

Einführung

Diese Aufgabe markiert den Wechsel von relationalen Datenbanken (SQL) hin zu einer flexiblen, dokumentenbasierten Speicherung (NoSQL) mit **MongoDB**. Für das Backend wird wieder Java **SpringBoot** verwendet.

Vorbereitung

Zuerst muss das Repo von dem Professor geklont bzw. in ein anderen Repo kopiert werden. Dafür das Repo vom Professor klonen und dann:

```
git remote remove origin

git remote add origin
https://github.com/ramisDayiniz/Ekici_DEZSYS_GK81_WAREHOUSE_ORM.git

git push -f --set-upstream origin main
```

Ein Docker **MongoDB**:

```
docker pull mongo

docker run -d -p 27017:27017 --name mongo-warehouse mongo
```

Fragestellungen

Hier sind die Antworten für dein Protokoll, kurz und prägnant formuliert:

4 Vorteile von NoSQL gegenüber RDBMS

1. **Flexibilität:** Kein festes Schema; Felder können jederzeit hinzugefügt werden.
2. **Skalierbarkeit:** Einfache horizontale Skalierung (Sharding) über viele Server.
3. **Performance:** Hohe Schreibgeschwindigkeiten und schnelle Abfragen bei hierarchischen Daten.
4. **Natürliches Datenformat:** Speicherung direkt als JSON/Dokument (passend zu modernen Programmiersprachen).

4 Nachteile von NoSQL gegenüber RDBMS

1. **Keine komplexen Joins:** Verknüpfungen zwischen Tabellen sind schwierig oder nicht vorhanden.
2. **Geringere Konsistenz:** Oft nur "Eventual Consistency" statt strikter ACID-Transaktionen.
3. **Datenredundanz:** Informationen werden oft mehrfach gespeichert (Denormalisierung).
4. **Fehlende Standardisierung:** Jede NoSQL-Datenbank hat eine eigene Abfragesprache (kein einheitliches SQL).

Schwierigkeiten bei der Zusammenführung (Aggregation)

- **Keine Joins:** Daten müssen händisch im Applikationscode oder über komplexe Aggregation-Pipelines kombiniert werden.
- **Inkonsistenz:** Daten könnten an einer Stelle aktualisiert worden sein, an einer anderen (redundanten) Stelle aber noch veraltet sein.

Arten von NoSQL-Datenbanken & Vertreter

Art	Vertreter
Dokumentenorientiert	MongoDB
Key-Value Store	Redis
Spaltenorientiert	Apache Cassandra
Graph-Datenbank	Neo4j

CAP-Theorem Abkürzungen

Das CAP-Theorem besagt, dass ein verteiltes System nur 2 dieser 3 Eigenschaften gleichzeitig garantieren kann:

- **CA (Consistency & Availability):** Daten sind überall gleich und das System antwortet immer (schwer in verteilten Netzen).
- **CP (Consistency & Partition Tolerance):** Daten sind überall gleich und das System überlebt Netzwerkausfälle, ist dann aber evtl. kurzzeitig nicht verfügbar (Beispiel: **MongoDB**).
- **AP (Availability & Partition Tolerance):** Das System ist immer verfügbar und überlebt Ausfälle, zeigt aber eventuell kurzzeitig veraltete Daten an.

MongoDB Shell Befehle (Abtestat)

1. Lagerstand eines Produkts über ALLER Lager anzeigen:

JavaScript

```
db.warehouses.aggregate([
  { $unwind: "$products" },
  { $match: { "products.productID": "P-1-10" } },
  { $project: { _id: 0, warehouseID: 1, name: 1, "products.productQuantity": 1 } }
])
```

2. Lagerstand eines Produkts eines BESTIMMTEN Lagers anzeigen:

JavaScript

```
db.warehouses.find(
  { "warehouseID": "WH-01", "products.productID": "P-1-10" },
  { _id: 0, "products.$": 1 }
)
```

Praxis

Model

Wie immer brauchen wir Model klassen für das Logik. Diese sind folgende:

Productdata

Diese Klasse sind Objekten für die Produkten selber.

```
package warehouse.model;

import java.io.Serializable;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.springframework.data.annotation.Id;

public class ProductData {

    @Id
    private String ID;

    private String warehouseID;
    private String productID;
    private String productName;
    private String productCategory;
    private double productQuantity;

    /**
     * Constructor
     */
    public ProductData() {
    }

    public ProductData(String warehouseID, String productID, String productName,
String productCategory, double productQuantity ) {
        super();
        this.warehouseID = warehouseID;
        this.productID = productID;
        this.productName = productName;
        this.productCategory = productCategory;
        this.productQuantity = productQuantity;
    }
    /**
     * Methods
     */
    @Override
    public String toString() {
        String info = String.format("Product Info: WarehouseID = %s, ProductID = %s,
ProductName = %s, ProductCategory = %s, ProductQuantity = %4.1f",
            warehouseID, productID, productName, productCategory, productQuantity );
        return info;
    }

    // Getter und Setter
}
```

Warehouse

Diese Klasse sind die Lager Objekten selber.

```
@Document(collection = "warehouses")
public class Warehouse {

    @Id
    private String id;

    private String warehouseID;
    private String name;
    private String city;
    private List<ProductData> products = new ArrayList<>();

    public Warehouse() {}

    public Warehouse(String warehouseID, String name, String city) {
        this.warehouseID = warehouseID;
        this.name = name;
        this.city = city;
    }

    public void addProduct(ProductData product) {
        this.products.add(product);
    }

    // Getter und Setter
}
```

Repository

Um Methoden zu erstellen von Model, Datenbank usw. Daten zu nehmen und diese zubearbeiten, brauchen wir eine RepositoryKlasse. Diesmal verwenden wir als Interface oder Klasse MongoRepository. Aber das Verwenden und die Benutzung der Repositories sind identisch.

```
public interface WarehouseRepository extends MongoRepository<Warehouse, String> {

//    public ProductData findByProductID(String productID);


    // Findet ein komplettes Lager-Dokument anhand der Geschäfts-ID (z.B. "WH-01")
    Warehouse findByWarehouseID(String warehouseID);

    // Falls du Lager nach Stadt suchen willst (optional für Berichte)
//    List<Warehouse> findByCity(String city);
}
```

Controller

Nach den Model Klassen brauchen wir natürlich für Rest Schnittstellen noch eine Controller Klasse, wo wir alle REST-Funktionen implementieren.

```java
@RestController
public class WarehouseController {

    @Autowired
    private WarehouseRepository repository;

    @PostMapping("/warehouse")
    public Warehouse addWarehouse(@RequestBody Warehouse warehouse) {
        return repository.save(warehouse);
    }

    @GetMapping("/warehouse")
    public List<Warehouse> getAllWarehouses() {
        return repository.findAll();
    }

    @GetMapping("/warehouse/{id}")
    public Warehouse getWarehouseById(@PathVariable String id) {
        return repository.findByWarehouseID(id);
    }

    @DeleteMapping("/warehouse/{id}")
    public void deleteWarehouse(@PathVariable String id) {
        Warehouse wh = repository.findByWarehouseID(id);
        if (wh != null) repository.delete(wh);
    }

    // --- PRODUCT ENDPOINTS ---

    // POST /product: Fügt ein Produkt zu einem Lager hinzu (Lager-ID muss im
ProductData Objekt sein)
    @PostMapping("/product")
    public Warehouse addProductToWarehouse(@RequestBody ProductData product) {
        Warehouse wh = repository.findByWarehouseID(product.getWarehouseID());
        if (wh != null) {
            wh.addProduct(product);
            return repository.save(wh);
        }
        return null; // Oder Error Handling
    }

    // GET /product: Alle Produkte über alle Lager hinweg finden
    @GetMapping("/product")
    public List<ProductData> getAllProducts() {
        return repository.findAll().stream()
                .flatMap(wh -> wh.getProducts().stream())
                .collect(Collectors.toList());
    }

    // GET /product/{id}: Ein Produkt anhand seiner ID suchen (über alle Lager)
```

```
    @GetMapping("/product/{id}")
    public List<ProductData> getProductByIdAcrossWarehouses(@PathVariable String id) {
        return repository.findAll().stream()
                .flatMap(wh -> wh.getProducts().stream())
                .filter(p -> p.getProductID().equals(id))
                .collect(Collectors.toList());
    }

    // DELETE /product/{id}: Löscht ein Produkt mit ID {id} aus ALLEN Lagern
    @DeleteMapping("/product/{id}")
    public void deleteProductFromAllWarehouses(@PathVariable String id) {
        List<Warehouse> allWarehouses = repository.findAll();
        for (Warehouse wh : allWarehouses) {
            boolean removed = wh.getProducts().removeIf(p ->
p.getProductID().equals(id));
            if (removed) {
                repository.save(wh);
            }
        }
    }
}
```

Application

Schließlich brauchen wir natürlich eine klasse, wo wir SpringBoot ausführen können und natürlich die Testdaten reinspeichern können.

```java
@SpringBootApplication
public class Application implements CommandLineRunner {

    @Autowired
    private WarehouseRepository repository;

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }


    @Override
    public void run(String... args) throws Exception {
        repository.deleteAll(); // Datenbank aufräumen

        String[] cities = {"Linz", "Wien", "Graz", "Salzburg", "Innsbruck"};
        String[] categories = {"Getraenk", "Waschmittel", "Tierfutter", "Reinigung",
"Elektronik", "Obst"};

        for (int i = 1; i <= 5; i++) {
            Warehouse wh = new Warehouse("WH-0" + i, "Lager " + cities[i-1], cities[i-
1]);

            // 60 Produkte pro Lager = 300 insgesamt
            for (int p = 1; p <= 60; p++) {
                String pId = "P-" + i + "-" + p;
                String pName = categories[p % categories.length] + " Artikel " + p;
                double qty = Math.random() * 500; // Zufälliger Lagerbestand

                wh.addProduct(new ProductData(wh.getWarehouseID(), pId, pName,
categories[p % categories.length], qty));
            }
            repository.save(wh);
        }

        System.out.println("Vertiefung: 5 Lager und 300 Produkte wurden erstellt!");
    }
}
```

Ergebnisse

```
2026-04-01T21:32:30.921+02:00  INFO 22572 --- [           main] warehouse.Application                    : Starti
2026-04-01T21:32:30.925+02:00  INFO 22572 --- [           main] warehouse.Application                    : No act
2026-04-01T21:32:31.425+02:00  INFO 22572 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootst
2026-04-01T21:32:31.466+02:00  INFO 22572 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finish
2026-04-01T21:32:31.793+02:00  INFO 22572 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat
2026-04-01T21:32:31.802+02:00  INFO 22572 --- [           main] o.apache.catalina.core.StandardService   : Starti
2026-04-01T21:32:31.802+02:00  INFO 22572 --- [           main] o.apache.catalina.core.StandardEngine    : Starti
2026-04-01T21:32:31.868+02:00  INFO 22572 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initia
2026-04-01T21:32:31.868+02:00  INFO 22572 --- [           main] w.s.c.ServletWebServerApplicationContext : Root W
2026-04-01T21:32:32.085+02:00  INFO 22572 --- [           main] org.mongodb.driver.client                : MongoC
2026-04-01T21:32:32.104+02:00  INFO 22572 --- [localhost:27017] org.mongodb.driver.cluster               : Monito
2026-04-01T21:32:32.672+02:00  INFO 22572 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat
2026-04-01T21:32:32.686+02:00  INFO 22572 --- [           main] warehouse.Application                    : Starte
Vertiefung: 5 Lager und 300 Produkte wurden erstellt!
```

http://localhost:8080/warehouse

GET http://localhost:8080/warehouse

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results — Status: 200 OK Time: 439 ms Size: 46.82 KB

Pretty Raw Preview Visualize JSON


```
[
    {
        "id": "69cd72d08f34bb1e4a55a9f7",
        "warehouseID": "WH-01",
        "name": "Lager Linz",
        "city": "Linz",
        "products": [
            {
                "warehouseID": "WH-01",
                "productID": "P-1-1",
                "productName": "Waschmittel Artikel 1",
                "productCategory": "Waschmittel",
                "productQuantity": 330.5269323832797,
                "id": null
            },
            {
                "warehouseID": "WH-01",
                "productID": "P-1-2",
                "productName": "Tierfutter Artikel 2",
                "productCategory": "Tierfutter",
                "productQuantity": 312.75704366366875,
                "id": null
            },
```


http://localhost:8080/product

GET http://localhost:8080/product

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Status: 200 OK Time: 134 ms Size: 46.3 KB

Pretty Raw Preview Visualize JSON


```
[
    {
        "warehouseID": "WH-01",
        "productID": "P-1-1",
        "productName": "Waschmittel Artikel 1",
        "productCategory": "Waschmittel",
        "productQuantity": 330.5269323832797,
        "id": null
    },
    {
        "warehouseID": "WH-01",
        "productID": "P-1-2",
        "productName": "Tierfutter Artikel 2",
        "productCategory": "Tierfutter",
        "productQuantity": 312.75704366366875,
        "id": null
    },
    {
        "warehouseID": "WH-01",
        "productID": "P-1-3",
        "productName": "Reinigung Artikel 3",
        "productCategory": "Reinigung",
        "productQuantity": 172.77665780599716,
        "id": null
    },
```


http://localhost:8080/warehouse/WH-01

GET http://localhost:8080/warehouse/WH-01

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Status: 200 OK Time: 203 ms Size: 9.49 KB

Pretty Raw Preview Visualize JSON


```
{
    "id": "69cd72d08f34bb1e4a55a9f7",
    "warehouseID": "WH-01",
    "name": "Lager Linz",
    "city": "Linz",
    "products": [
        {
            "warehouseID": "WH-01",
            "productID": "P-1-1",
            "productName": "Waschmittel Artikel 1",
            "productCategory": "Waschmittel",
            "productQuantity": 330.5269323832797,
            "id": null
        },
        {
            "warehouseID": "WH-01",
            "productID": "P-1-2",
            "productName": "Tierfutter Artikel 2",
            "productCategory": "Tierfutter",
            "productQuantity": 312.75704366366875,
            "id": null
        },
```


Unterschied der Endpunkte

- `/warehouse` **(Dokumenten-Sicht):** Zeigt das Lager als **Hauptobjekt**. Die Produkte sind darin "eingebettet" (verschachtelt). Das entspricht der echten Speicherung in MongoDB.
- `/product` **(Listen-Sicht):** Zeigt eine **flache Liste** aller Produkte. Deine Middleware (Java) hat die Verschachtelung aufgelöst, damit man alle Artikel direkt untereinander sehen kann.

MongoBash

```
PS C:\Users\ekici> docker exec -it mongo-warehouse mongosh use mongo-warehouse
Error: ENOENT: no such file or directory, open '/mongo-warehouse'
PS C:\Users\ekici> docker exec -it mongo-warehouse mongosh
Current Mongosh Log ID: 69cd6e5da0d4e397f044ba88
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&server
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2
```

Teil 1: CRUD

Dokumentiere diese Befehle und mach jeweils einen Screenshot vom Ergebnis.

1. **Create (Ein Produkt zu einem Lager hinzufügen):**

```
db.warehouses.updateOne(
 { "warehouseID": "WH-01" },
 { $push: { "products": { "productID": "P-999", "productName": "Shell-Test",
"productCategory": "Test", "productQuantity": 100 } } }
)
```

2. **Read (Ein bestimmtes Lager finden):**

```
db.warehouses.find({ "warehouseID": "WH-01" }).pretty()
```

3. **Update (Die Menge eines Produkts ändern):**

```
db.warehouses.updateOne(
 { "warehouseID": "WH-01", "products.productID": "P-999" },
 { $set: { "products.$.productQuantity": 150 } }
)
```

4. **Delete (Ein Produkt aus einem Lager löschen):**

```
db.warehouses.updateOne(
 { "warehouseID": "WH-01" },
 { $pull: { "products": { "productID": "P-999" } } }
)
```

5. **Delete (Ein ganzes Lager löschen):**

```
db.warehouses.deleteOne({ "warehouseID": "WH-05" })
```

1.

```
test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   40.00 KiB
test    88.00 KiB
test> use test
already on db test
test> show collections
warehouses
test> db.warehouses.updateOne(
|   { "warehouseID": "WH-01" },
|   { $push: { "products": { "productID": "P-999", "productName": "Shell-Test", "productCategory": "Test", "productQuant
ity": 100 } } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

2.

```
test> db.warehouses.find({ "warehouseID": "WH-01" }).pretty()
[
  {
    _id: ObjectId('69cd6e78cc5329442d077c7d'),
    warehouseID: 'WH-01',
    name: 'Lager Linz',
    city: 'Linz',
    products: [
      {
        warehouseID: 'WH-01',
        productID: 'P-1-1',
        productName: 'Waschmittel Artikel 1',
        productCategory: 'Waschmittel',
        productQuantity: 42.979099978482424
      },
      {
        warehouseID: 'WH-01',
        productID: 'P-1-2',
        productName: 'Tierfutter Artikel 2',
        productCategory: 'Tierfutter',
        productQuantity: 89.47858284507
      },
```

3.

```
test> db.warehouses.updateOne(
|   { "warehouseID": "WH-01", "products.productID": "P-999" },
|   { $set: { "products.$.productQuantity": 150 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

4.

```
test> db.warehouses.updateOne(
|   { "warehouseID": "WH-01" },
|   { $pull: { "products": { "productID": "P-999" } } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
test>
```

5.

```
test> db.warehouses.deleteOne({ "warehouseID": "WH-05" })
{ acknowledged: true, deletedCount: 1 }
test>
```

Teil 2: 3 Berichts-Fragestellungen (Vertiefung)

Das ist der "intelligente" Teil für die Note 1.

Fragestellung 1: Wie ist der Lagerbestand eines Produkts über ALLE Standorte? *Hilfreich, um zu sehen, ob man intern Ware verschieben kann.*

```
db.warehouses.aggregate([
  { $unwind: "$products" },
  { $match: { "products.productID": "P-1-10" } },
  { $group: { _id: "$products.productName", Gesamtbestand: { $sum:
"$products.productQuantity" } } }
])
```

Fragestellung 2: Welche Produkte haben einen kritischen Bestand (unter 10 Stück) über alle Lager? *Wichtig für den Einkauf, um nachzubestellen.*

```
db.warehouses.aggregate([
  { $unwind: "$products" },
  { $match: { "products.productQuantity": { $lt: 10 } } },
  { $project: { _id: 0, Lager: "$name", Produkt: "$products.productName", Menge:
"$products.productQuantity" } }
])
```

Fragestellung 3: Wie viele Produkte jeder Kategorie haben wir insgesamt im System? *Übersicht für das Management.*

```
db.warehouses.aggregate([
  { $unwind: "$products" },
  { $group: { _id: "$products.productCategory", Anzahl_Produkte: { $count: {} } } }
])
```

Fragestellung 1.

```
test> db.warehouses.aggregate([
|   { $unwind: "$products" },
|   { $match: { "products.productID": "P-1-10" } },
|   { $group: { _id: "$products.productName", Gesamtbestand: { $sum: "$products.productQuantity" } } }
| ])
[ { _id: 'Elektronik Artikel 10', Gesamtbestand: 490.8886030759108 } ]
test>
```

Fragestellung 2.

```
test> db.warehouses.aggregate([
|   { $unwind: "$products" },
|   { $match: { "products.productQuantity": { $lt: 10 } } },
|   { $project: { _id: 0, Lager: "$name", Produkt: "$products.productName", Menge: "$products.productQuantity" } }
| ])
[
  {
    Lager: 'Lager Linz',
    Produkt: 'Tierfutter Artikel 14',
    Menge: 2.15282132108241
  },
  {
    Lager: 'Lager Linz',
    Produkt: 'Waschmittel Artikel 19',
    Menge: 1.2375120452486343
  },
  {
    Lager: 'Lager Wien',
    Produkt: 'Tierfutter Artikel 26',
    Menge: 0.764740008657061
```

Fragestellung 3.

```
test> db.warehouses.aggregate([
|   { $unwind: "$products" },
|   { $group: { _id: "$products.productCategory", Anzahl_Produkte: { $count: {} } } }
| ])
[
  { _id: 'Elektronik', Anzahl_Produkte: 40 },
  { _id: 'Reinigung', Anzahl_Produkte: 40 },
  { _id: 'Waschmittel', Anzahl_Produkte: 40 },
  { _id: 'Tierfutter', Anzahl_Produkte: 40 },
  { _id: 'Getraenk', Anzahl_Produkte: 40 },
  { _id: 'Obst', Anzahl_Produkte: 40 }
]
test>
```